

Câu 1: (2.5 đ)

Thiết kế mạch giải mã địa chỉ và kết nối dữ liệu cho hệ thống (8-bit địa chỉ A7-A0; 8-bit dữ liệu D7-D0; RD, WR trên 2 chân riêng biệt) kết nối với các module bên dưới. Ghi rõ kết nối chân CS, WR, RD tới các module và địa chỉ bắt đầu, địa chỉ kết thúc của các module (không giải mã địa chỉ các kênh):

- 1/ Module 1: 10 kênh ADC 16-bit
- 2/ Module 2: 2 kênh DAC 12-bit
- 3/ Module 3: 6 kênh PWM 12-bit
- 4/ Module 4: 6 kênh Encoder 24-bit
- 5/ Module 5: 8 kênh Digital Input 8-bit
- 6/ Module 6: 8 kênh Digital Output 8-bit

Câu 2: (2đ)

Giải thích và tính **a, b, y** ở thời điểm ban đầu và sau 3 xung clock trong 2 chương trình:

```
output [7:0] y;  
reg [7:0] a = 1;  
reg [7:0] b = 2;  
reg [1:0] sel = 0;  
  
always @(posedge clock)  
begin  
    sel <= sel + 1;  
    a <= b + 5;  
    b <= a * 2;  
end  
assign y = (sel==0)? 8'h80:  
            (sel==1)? a:  
            (sel==2)? b:  
            (sel==3)? 0:8'b0;
```

```
reg [7:0] y = 0;  
reg [7:0] a = 1;  
reg [7:0] b = 2;  
reg [1:0] sel = 0;  
  
always @(posedge clock)  
begin  
    sel = sel + 1;  
    b = a + 5;  
    a = sel * 2;  
    y = (a<1) & (b<1);  
end
```

Câu 3: (2.5 đ)

Viết chương trình Verilog thực hiện xuất tín hiệu điều khiển đèn giao thông như sau:

- Tên module: Den_giao_thong (clk, Tred, Tyellow, Tgreen, R1, Y1, G1, R2, Y2, G2)
- Tín đầu vào:
 - o clk: tín hiệu xung clock 1us.
 - o Tred, Tyellow, Tgreen: thời gian sáng đèn đỏ, đèn vàng, đèn xanh (đơn vị: s)
Tred = Tyellow + Tgreen
- Tín hiệu đầu ra:
 - o R1, Y1, G1: tín hiệu đèn đỏ, đèn vàng, đèn xanh trên trụ đèn 1.
 - o R2, Y2, G2: tín hiệu đèn đỏ, đèn vàng, đèn xanh trên trụ đèn 2.

Câu 4: (3 đ)

Trong yêu cầu thiết kế hệ thống dùng STM32F4 ta cần sử dụng Port PB[15:6] cho 4 chức năng SPI, I2C, UART và CAN. Hãy cấu hình các thanh ghi hệ thống để sử dụng 4 chức năng này trên các chân từ PB6 đến PB15 (chú ý không được dùng các chân khác) với các yêu cầu sau:

- a. Chỉ rõ chân nào sử dụng SPI, I2C, UART, CAN
- b. Cấu hình các thanh ghi của vi điều khiển để 4 chức năng trên hoạt động (Cấu hình thanh ghi cho phép xung clock để hoạt động 4 chức năng trên, cấu hình thanh ghi chân IO để lựa chọn chức năng phù hợp)

Câu 1: (2,5 đ)

Thiết kế mạch giải mã địa chỉ và kết nối dữ liệu cho hệ thống (8-bit địa chỉ A7-A0; 8-bit dữ liệu D7-D0; RD, WR trên 2 chân riêng biệt) kết nối với các module bên dưới. Ghi rõ kết nối chân CS, WR, RD tới các module và địa chỉ bắt đầu, địa chỉ kết thúc của các module (không ghi chi tiết địa chỉ các kênh):

- 1/ Module 1: 12 kênh ADC 16-bit
- 2/ Module 2: 2 kênh DAC 12-bit
- 3/ Module 3: 6 kênh PWM 10-bit
- 4/ Module 4: 6 kênh Encoder 32-bit
- 5/ Module 5: 2 kênh Digital Input 8-bit

Câu 2: (2,5 đ)

Để đo tốc độ động cơ ta thường thực hiện đếm số xung tín hiệu encoder ENCA trong 1 khoảng thời gian cố định. Viết chương trình Verilog thực hiện đếm số xung ENCA trong 10ms với các yêu cầu sau:

- Đầu vào tín hiệu xung cần đo: ENCA
- Đầu vào tín hiệu xung clock 1us: clk
- Đầu ra dữ liệu 16 bit: D[15:0] (đơn vị: số xung / 10ms)
- Dữ liệu ngõ ra được cập nhật sau mỗi 10ms

Gợi ý: Tạo 1 bộ chia tần số từ xung clk (1us) để có tín hiệu xung 10ms, sau đó đầu ra D[15:0] sẽ được cập nhật theo tín hiệu xung 10ms này.

Câu 3: (2,5 đ)

Trong yêu cầu thiết kế hệ thống dùng STM32F4 ta cần sử dụng Port PD[7:0] cho 2 chức năng UART và CAN. Hãy cấu hình các thanh ghi hệ thống để sử dụng 2 chức năng này trên các chân từ PD0 đến PD7 (chú ý không được dùng các chân khác) với các yêu cầu sau:

- Chỉ rõ chân nào sử dụng UART, chân nào sử dụng CAN.
- Cấu hình thanh ghi cho phép xung clock để hoạt động 2 chức năng trên.
- Cấu hình thanh ghi chân IO để lựa chọn chức năng phù hợp.

Câu 4: (2,5 đ)

Trình bày ý tưởng và nội dung một đề tài có sử dụng Hệ thống điều khiển nhúng với các yêu cầu sau:

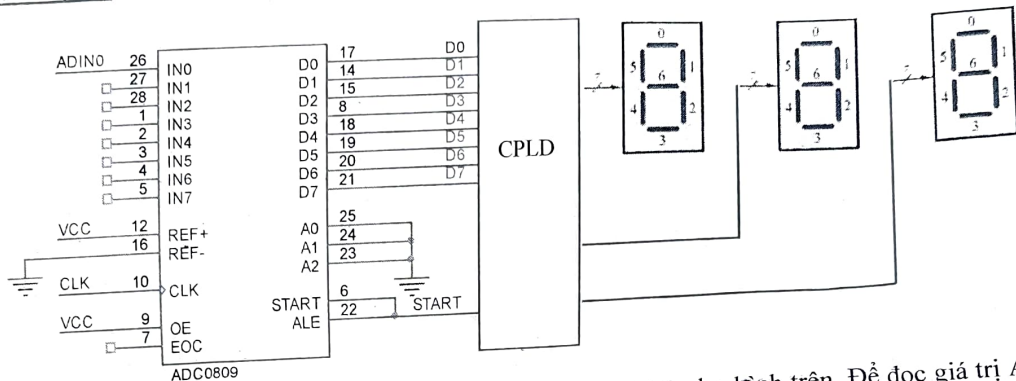
1. Mục tiêu của đề tài.
2. Nêu nội dung và giải pháp để thực hiện ý tưởng đề tài đặt ra.
3. Sơ đồ chân kết nối tín hiệu từ Vi điều khiển tới các khối chức năng.

Câu 1: (3 đ)

Thiết kế mạch giải mã địa chỉ và kết nối dữ liệu cho hệ thống (**8-bit địa chỉ A7-A0; 16-bit dữ liệu D15-D0; RD, WR trên 2 chân riêng biệt**) kết nối với các module bên dưới. Ghi rõ kết nối chân CS, WR, RD tới các module và địa chỉ bắt đầu, địa chỉ kết thúc của các module (không giải mã địa chỉ các kênh):

- 1/ Module 1: 8 kênh ADC 16-bit
- 2/ Module 2: 2 kênh DAC 12-bit
- 3/ Module 3: 12 kênh PWM 12-bit
- 4/ Module 4: 12 kênh Encoder 32-bit
- 5/ Module 5: 4 kênh Digital Input 8-bit

Câu 2: (3 đ)



Cho sơ đồ kết nối CPLD với ADC0809 để đọc kênh ADIN0 như hình trên. Để đọc giá trị ADC đầu tiên ta kích xung START với độ rộng xung 10us, sau thời gian 100us dữ liệu ADC sẽ có ở D7-D0. Viết chương trình verilog đọc giá trị ADC và hiển thị giá trị ra 3 LED bảy đoạn (cathode chung) với các yêu cầu sau:

- Module **adc_read**(clk, D, Start, LED_TR, LED_CH, LED_DV)
- Đầu vào tín hiệu xung clock 1us: clk
- Đầu vào dữ liệu 8bit từ bộ biến đổi ADC: D[7:0]
- Đầu ra tín hiệu kích biến đổi ADC: START.
- Đầu ra dữ liệu của 3 LED bảy đoạn: LED_TR[6:0], LED_CH[6:0], LED_DV[6:0].
- Chu kỳ kích xung START và hiển thị ra LED là 50ms.

Câu 3: (2 đ)

Một hệ thống định vị tích hợp GPS/INS sử dụng 2 cảm biến ADIS16407 (giao tiếp SPI ở PB12-PB15) và GPS (giao tiếp UART ở PC10-PC11). Vi điều khiển STM32F405 thực hiện thuật toán ước lượng và xuất dữ liệu ra cổng UART2 ở PA2-PA3. Ngoài ra, dữ liệu được lưu trữ trong thẻ nhớ thông qua giao tiếp SPI ở PB3-PB5, PA15.

Hãy cấu hình các thanh ghi của vi điều khiển để 4 khối trên (2 SPI và 2 UART) hoạt động.

Câu 4: (2 đ)

Trong ứng dụng hệ thống kết hợp STM32F4 và CPLD ta cần kết nối 2 thiết bị theo kiểu giải mã địa chỉ (chức năng FSMC trên STM32F4) với các chân: Data[15:0], Addr[7:0], CS(NE1), RD, WR.

- a. Vẽ sơ đồ kết nối trên STM32F4 tới CPLD trong đó ghi rõ chân nào trên STM32F4 kết nối với Data[15:0], Addr[7:0], CS(NE1), RD, WR của CPLD.
- b. Hãy cấu hình thanh ghi trên STM32F4 để cho phép xung clock hoạt động và cấu hình thanh ghi chân IO để lựa chọn chức năng FSMC.

Câu 1: (2,5đ)

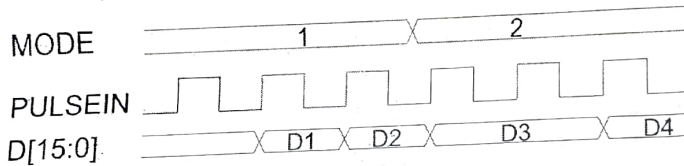
Thiết kế mạch giải mã địa chỉ và kết nối dữ liệu cho hệ thống (8-bit địa chỉ A7-A0; 8-bit dữ liệu D7-D0; RD, WR trên 2 chân riêng biệt) kết nối với các module bên dưới. Ghi rõ kết nối chân CS, WR, RD tới các module và địa chỉ bắt đầu, địa chỉ kết thúc của các module (không giải mã địa chỉ các kênh):

- | | |
|--------------------------------|--|
| 1/ Module 1: 6 kênh ADC 16-bit | 4/ Module 4: 8 kênh Encoder 16-bit |
| 2/ Module 2: 4 kênh DAC 12-bit | 5/ Module 5: 4 kênh Digital Input 8-bit |
| 3/ Module 3: 8 kênh PWM 10-bit | 6/ Module 6: 2 kênh Digital Output 8-bit |

Câu 2: (2,5đ)

Viết chương trình Verilog tên **doxung**(clk, PULSEIN, MODE, D) thực hiện việc đo chu kỳ của xung PULSEIN như sau:

- Đầu vào tín hiệu xung clock clk có tần số 1 MHz.
- Đầu vào tín hiệu PULSEIN, xung vuông với tần số thay đổi
- Đầu vào MODE[1:0], nếu MODE = 1: đo thời gian 1 chu kỳ xung, nếu MODE = 2: đo thời gian 2-chu kỳ xung như mô tả ở Hình 1. (MODE được cài đặt giá trị lúc ban đầu, không thay đổi trong quá trình chạy)
- Đầu ra dữ liệu 16 bit: D[15:0] (đơn vị us). Dữ liệu ngõ ra được cập nhật sau mỗi 1 chu kỳ hoặc 2 chu kỳ của PULSEIN tùy theo giá trị của MODE.



Hình 1. Giải đồ xung

Câu 3: (2,5đ)

Trong yêu cầu thiết kế hệ thống dùng STM32F4 ta cần sử dụng Port PB[15:8] cho chức năng UART và SPI, Port PC[3:0] cho chức năng ADC 4 kênh. Hãy cấu hình các thanh ghi hệ thống để sử dụng 3 chức năng này trên các chân từ PB8-PB15 và PC0-PC3 (chú ý không được dùng các chân khác) với các yêu cầu sau:

- Chỉ rõ chân nào sử dụng UART, chân nào sử dụng SPI, chân nào sử dụng kênh ADC
- Cấu hình thanh ghi cho phép xung clock để hoạt động 3 chức năng trên
- Cấu hình thanh ghi chân IO để lựa chọn chức năng phù hợp

Câu 4: (2,5đ)

Trong cấu hình kết nối ở Câu 3, các hàm đọc giá trị ADC và gửi dữ liệu ra UART được cung cấp như sau:

- Hàm đọc các kênh ADC: unsigned int **read_adc** (int channel). Trong đó *channel* có giá trị từ 1 đến 4 tương ứng với 4 kênh.
- Hàm gửi N byte từ bộ đệm *txbuff* ra UART: void **send_data** (char *txbuff, int N)

Viết chương trình đọc tất cả 4 kênh ADC, sau đó chuyển đổi giá trị các kênh sang ASCII và gửi ra UART theo format chuỗi dữ liệu như sau:

ADC 1 (5byte)	<space> (1byte)	ADC 2 (5byte)	<space> (1byte)	ADC 3 (5byte)	<space> (1byte)	ADC 4 (5byte)	0x0D (1byte)	0x0A (1byte)
------------------	--------------------	------------------	--------------------	------------------	--------------------	------------------	-----------------	-----------------

Câu 1: (2,5đ)

Thiết kế mạch giải mã địa chỉ và kết nối dữ liệu cho hệ thống (8-bit địa chỉ A7-A0; 8-bit dữ liệu D7-D0; RD, WR trên 2 chân riêng biệt) kết nối với các module bên dưới. Ghi rõ kết nối chân CS, WR, RD tới các module và địa chỉ bắt đầu, địa chỉ kết thúc của các module (không giải mã địa chỉ các kênh):

- 1/ Module 1: 16 kênh ADC 16-bit
- 2/ Module 2: 4 kênh DAC 12-bit
- 3/ Module 3: 16 kênh PWM 10-bit

- 4/ Module 4: 16 kênh Encoder 32-bit
- 5/ Module 5: 4 kênh Digital Input 8-bit
- 6/ Module 6: 2 kênh Digital Output 8-bit

Câu 2: (2,5đ)

Viết chương trình Verilog tên **demxung**(clk, MODE, PulseIn, D) thực hiện việc đếm xung kênh PulseIn như sau:

- Đầu vào tín hiệu xung clock clk có tần số 1 MHz.
- Đầu vào tín hiệu PulseIn có tần số thay đổi
- Đầu vào MODE, nếu MODE = 0: đếm số xung trong thời gian 10ms, nếu MODE = 1: đếm số xung trong thời gian 1s. (MODE được cài đặt giá trị lúc ban đầu, không thay đổi trong quá trình chạy)
- Đầu ra dữ liệu 16 bit: D[15:0] (đơn vị: số xung). Dữ liệu ngõ ra được cập nhật sau mỗi chu kỳ 10ms hoặc 1s tùy theo giá trị của MODE.

Câu 3: (2,5đ)

Trong yêu cầu thiết kế hệ thống dùng STM32F4 ta cần sử dụng Port PD[7:0] cho chức năng UART và CAN, Port PA[5:4] cho chức năng DAC 2 kênh. Hãy cấu hình các thanh ghi hệ thống để sử dụng 3 chức năng này trên các chân từ PD0-PD7 và PA4-PA5 (chú ý không được dùng các chân khác) với các yêu cầu sau:

- Chỉ rõ chân nào sử dụng UART, chân nào sử dụng CAN, chân nào sử dụng 2 kênh DAC
- Cấu hình thanh ghi cho phép xung clock để hoạt động 3 chức năng trên
- Cấu hình thanh ghi chân IO để lựa chọn chức năng phù hợp

Câu 4: (2,5đ)

Trong cấu hình kết nối ở Câu 3, các hàm xuất giá trị DAC và nhận dữ liệu từ UART được cung cấp như sau:

- Hàm xuất các kênh DAC: void **write_dac** (int *channel*, int *value*). Trong đó *channel* có giá trị từ 1 đến 2 tương ứng với 2 kênh.
- Hàm nhận N byte từ bộ đệm *rxbuff* qua UART: void **receive_data** (char **rxbuff*, int *N*). Biết giá trị DAC dạng ASCII 4 bytes và format chuỗi dữ liệu nhận vào như sau:

DAC 1	<space>	DAC 2	0x0D	0x0A
(4byte)	(1byte)	(4byte)	(1byte)	(1byte)

Viết chương trình đọc dữ liệu từ UART, sau đó chuyển đổi giá trị ASCII sang dạng int và xuất ra 2 kênh DAC.

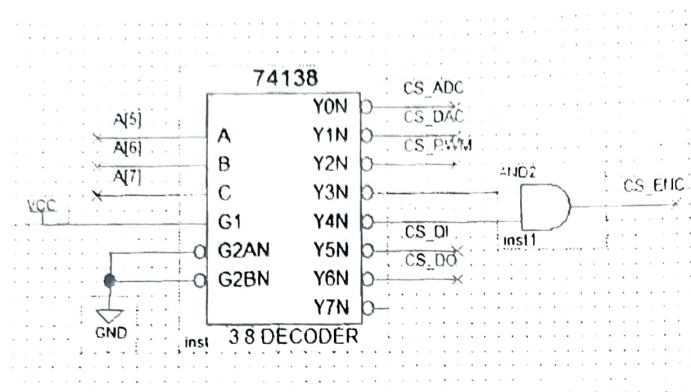
CNBM

TS. Nguyễn Vinh Hào

ĐÁP ÁN

Câu 1: (2,5đ)

- Ghi tâm địa chỉ đúng cho 6 module và sơ đồ kết nối 74138 (1,5đ)



- Vẽ sơ đồ giải mã đúng 6 module (1đ)

Câu 2: (2,5đ)

```
module demxung (clk, MODE, PulseIn, D);
input clk, MODE, PulseIn;
output reg [15:0] D;
reg [15:0] temp = 0;
reg [21:0] cnt = 0;
reg prePulseIn = 0;
```

```
always @ (posedge clk) begin
prePulseIn <= PulseIn;
if ({prePulseIn, PulseIn} == 2'b01)
temp = temp + 1;
```

```
cnt = cnt + 1;
if ((cnt==10000) && (MODE == 0)) begin
cnt = 0;
D = temp;
temp = 0;
end
else if ((cnt==1000000) && (MODE == 1)) begin
cnt = 0;
D = temp;
temp = 0;
end
end
```

```
end
endmodule
```

Câu 2: (2,5đ)

Viết chương trình Verilog tên demxung(clk, MODE, PulseIn, D) thực hiện việc đếm xung kênh PulseIn như sau:

- Đầu vào tín hiệu xung clock clk có tần số 1 MHz.
- Đầu vào tín hiệu PulseIn có tần số thay đổi
- Đầu vào MODE, nếu MODE = 0: đếm số xung trong thời gian 10ms, nếu MODE = 1: đếm số xung trong thời gian 1s. (MODE được cài đặt giá trị lúc ban đầu, không thay đổi trong quá trình chạy)
- Đầu ra dữ liệu 16 bit: D[15:0] (đơn vị: số xung). Dữ liệu ngõ ra được cập nhật sau mỗi chu kỳ 10ms hoặc 1s tùy theo giá trị của MODE.

Đếm xung PulseIn

Cập nhật giá trị ngõ ra

Câu 3: (2,5đ)

- Chỉ rõ chân kết nối:

PD0 – PD1: CAN1
PD5 – PD6: UART2
PA4 – PA5: DAC

- Cấu hình thanh ghi xung clock

```
RCC_AHB1ENR |= (1<<0) | (1<<3);
RCC_APB1ENR |= (1<<17) | (1<<25);
RCC_AHB1ENR |= (1<<29);
```

(0,5đ)

(1đ)

```
//clock cho PA và PD
//clock cho UART2 và CAN1
//clock cho DAC
```

- Cấu hình mode chức năng

$\text{GPIO_MODER} = (2 \ll 0) | (2 \ll 2) | (2 \ll 10) | (2 \ll 12);$

(1d)

$\text{GPIOA_MODER} = (3 \ll 8) | (3 \ll 10);$

$\text{GPIO_AFRL} = (9 \ll 0) | (9 \ll 4) | (7 \ll 20) | (7 \ll 24);$

Câu 4: (2,5đ)

```
char rxbuff[11];
int16_t dacval1, dacval2;

main() {
    while(1) {
        receive(rxbuff, 11);
        if ((rxbuff[9] == 0x0D) && (rxbuff[10] == 0x0A)) {
            dacval1 = (rxbuff[0] - 48) * 1000 + (rxbuff[1] - 48) * 100 +
                      (rxbuff[2] - 48) * 10 + (rxbuff[3] - 48);
            dacval2 = (rxbuff[5] - 48) * 1000 + (rxbuff[6] - 48) * 100 +
                      (rxbuff[7] - 48) * 10 + (rxbuff[8] - 48);
            dac_write(1, dacval1);
            dac_write(2, dacval2);
        }
    }
}
```



TS. Nguyễn Vĩnh Hào

Câu 1: (2,5đ)

Thiết kế mạch giải mã địa chỉ và kết nối dữ liệu cho hệ thống (8-bit địa chỉ A7-A0; 16-bit dữ liệu D15-D0; RD, WR trên 2 chân riêng biệt) kết nối với các module bên dưới. Ghi rõ kết nối chân CS, WR, RD tới các module và địa chỉ bắt đầu, địa chỉ kết thúc của các module (không giải mã địa chỉ các kênh):

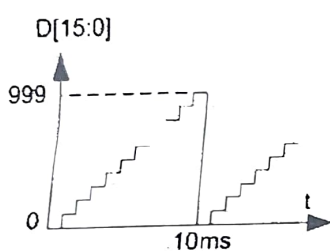
- 1/ Module 1: 10 kênh ADC 16-bit
- 2/ Module 2: 10 kênh DAC 12-bit
- 3/ Module 3: 16 kênh PWM 10-bit

- 4/ Module 4: 16 kênh Encoder 32-bit
- 5/ Module 5: 4 kênh Digital Input 8-bit
- 6/ Module 6: 2 kênh Digital Output 8-bit

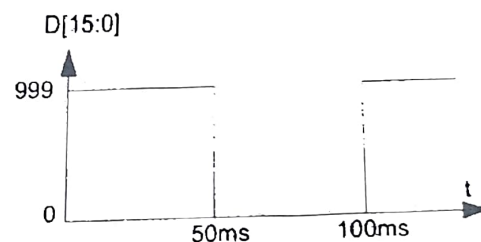
Câu 2: (2,5đ)

Viết chương trình Verilog tên **xuatxung(clk, Smode, Fmode, D)** thực hiện việc xuất giá trị xung tam giác và xung vuông với tần số thay đổi ra D[15:0] như sau:

- Đầu vào tín hiệu xung clock clk có tần số 1 MHz.
- Đầu vào Smode: chọn dạng sóng: Smode = 0: sóng tam giác, Smode = 1: sóng xung vuông
- Đầu vào Fmode: chọn mode tần số: Fmode = 0: tần số 100Hz, Fmode = 1: tần số 10Hz
- Đầu ra 16-bit D[15:0]: Giá trị thay đổi theo {Smode, Fmode} như minh họa 2 trong 4 trường hợp của {Smode, Fmode}: Hình 1 (Smode=0, Fmode=0) và Hình 2 (Smode=1, Fmode=1).



Hình 1. (Smode = 0, Fmode = 0)



Hình 2. (Smode = 1, Fmode = 1)

Câu 3: (2,5đ)

Trong yêu cầu thiết kế hệ thống dùng STM32F4 ta cần sử dụng Port PB[9:6] cho chức năng UART và CAN, Port PC[1:0] cho chức năng ADC 2 kênh. Hãy cấu hình các thanh ghi hệ thống để sử dụng 3 chức năng này trên các chân từ PB6-PB9 và PC0-PC1 (chú ý không được dùng các chân khác) với các yêu cầu sau:

- Chỉ rõ chân nào sử dụng UART, chân nào sử dụng CAN, chân nào sử dụng 2 kênh ADC
- Cấu hình thanh ghi cho phép xung clock để hoạt động 3 chức năng trên
- Cấu hình thanh ghi chân IO để lựa chọn chức năng phù hợp

Câu 4: (2,5đ)

Dựa cấu hình kết nối và giải mã địa chỉ ở Câu 1, cho trước hàm:

- Hàm gửi N byte từ bộ đệm *txbuff* ra UART: void **send_data** (char *txbuff, int N)

Viết chương trình đọc dữ liệu từ module Encoder, kênh 1, sau đó chuyển đổi giá trị int sang ASCII và gửi ra UART theo format chuỗi dữ liệu như sau biết rằng giá trị đọc encoder có tầm từ 0-100.000:

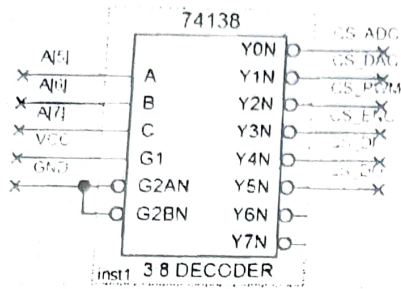
Encoder 1 (6byte)	0x0D (1byte)	0x0A (1byte)
----------------------	-----------------	-----------------

CNBM
TS. Nguyễn Vĩnh Khảo

ĐÁP ÁN

Câu 1: (2,5đ)

- Ghi tầm địa chỉ đúng cho 6 module và sơ đồ kết nối 74138 (1,5đ)



- Vẽ sơ đồ giải mã đúng 6 module

(1đ)

Câu 2: (2,5đ)

```
module xuattxung(clk, Smode, Fmode, D);
input clk, Smode, Fmode;
output reg [15:0] D;
reg [7:0] cnt = 0;
reg [15:0] cnt0 = 0;
```

- Xung clk 1MHz
- Fmode = 0 => Đếm 10000
- Fmode = 1 => Đếm 100000
- Giá trị sóng từ 0 đến 999 => Đếm 1000

```
wire [7:0] N;
assign N = Fmode? 99 : 9;

always @ (posedge clk) begin
    if (cnt < N)
        cnt = cnt + 1;
    else begin
        cnt = 0;
        if (cnt0 < 999)
            cnt0 = cnt0 + 1;
        else
            cnt0 = 0;

        if (!Smode)
            D = cnt0;
        else begin
            if (cnt0 < 500)
                D = 999;
            else
                D = 0;
        end
    end
end
```

- => Fmode = 0 => Đếm 10 => Cập nhật giá trị sóng
- => Fmode = 1 => Đếm 100 => Cập nhật giá trị sóng

- [Vàng] Đếm số xung để cập nhật giá trị
- [Lam] Kiểm tra giá trị đạt 1000 chưa
- [Lục] Xuất giá trị

```
end
end
endmodule
```

Câu 3: (2,5đ)

- Chỉ rõ chân kết nối:

(0.5đ)

PB6 – PB7: UART1
PB8 – PB9: CAN1
PC0 – PC1: ADC

- Cấu hình thành ghi xung clock

(1đ)

RCC_AHB1ENR |= (1<<1) | (1<<2); //clock cho PB và PC
RCC_APB1ENR |= (1<<25); //clock cho CAN1
RCC_APB2ENR |= (1<<4) | (1<<8) | (1<<9); //clock cho UART1, ADC

- Cấu hình mode chức năng

(1đ)

GPIOB_MODER |= (2<<12) | (2<<14) | (2<<16) | (2<<18);
GPIOC_MODER |= (3<<0) | (3<<2);
GPIOB_AFRL |= (7<<24) | (7<<28);
GPIOB_AFRH |= (9<<0) | (9<<4);

Câu 4: (2,5đ)

```
void IntToStr6(int32_t u, uint8_t *y){  
    int32_t a, i;  
  
    for (i=5; i>=0; i--){  
        y[i] = a % 10 + '0';  
        a = a / 10;  
    }  
}
```

(1đ)

```
main(){  
    int32_t enc_value;  
    uint16_t *p;  
    uint8_t txbuff[ ];  
  
    p = 0x61;     //theo địa chỉ giải mã ở câu 1 (MSB)  
    enc_value = *p--;  
    enc_value = enc_value<<16 + *p;  
  
    IntToStr6(enc_value, txbuff);  
    txbuff[6] = 0x0D;  
    txbuff[7] = 0x0A;  
    send_data(txbuff, 8);  
}
```

(1đ)

(0.5đ)

CNBH
Khach

TS. Nguyễn Vĩnh Hào



TRƯỜNG ĐH BÁCH KHOA – ĐHQG-HCM
KHOA ĐIỆN ĐIỆN TỬ

THI CUỐI KỲ

Học kỳ/năm học	1	2022-2023
Ngày thi	21/12/2022	

Môn học	Hệ thống điều khiển nhúng
Mã môn học	EE3067
Thời lượng	80 phút
Mã đề	

Ghi chú: - Được sử dụng tài liệu

Câu 1: (L.O.1) (2,5đ)

Thiết kế mạch giải mã địa chỉ và kết nối dữ liệu cho hệ thống (8-bit địa chỉ A7-A0; 16-bit dữ liệu D15-D0; RD, WR trên 2 chân riêng biệt) kết nối với các module bên dưới. Ghi rõ kết nối chân CS, WR, RD tới các module và địa chỉ bắt đầu, địa chỉ kết thúc của các module (không giải mã địa chỉ các kênh):

- 1/ Module 1: 16 kênh ADC 16-bit
- 2/ Module 2: 10 kênh DAC 12-bit
- 3/ Module 3: 16 kênh PWM 10-bit

- 4/ Module 4: 16 kênh Encoder 32-bit
- 5/ Module 5: 8 kênh Digital Input 8-bit
- 6/ Module 6: 4 kênh Digital Output 8-bit

Câu 2: (L.O.2) (2,5đ)

Viết chương trình Verilog thực hiện xuất tín hiệu xung với tần số thay đổi như sau:

- Tên module: xuất_xung (clk, PulseOut, mode)
- 2 đầu vào:
 - o tín hiệu xung clock 1us: clk.
 - o mode: chọn tần số xuất xung (2 bits)
 - mode = 0: xuất xung tần số 1 KHz
 - mode = 1: xuất xung tần số 0.2 KHz
 - mode = 2: xuất xung tần số 0.5 KHz
 - mode = 3: xuất xung tần số 0.1 Hz;
- Tín hiệu đầu ra: PulseOut.

Câu 3: (L.O.5) (2,5đ)

Trong yêu cầu thiết kế hệ thống dùng STM32F4 ta cần sử dụng Port {PD8, PD9} cho UART, {PE5, PE6} cho PWM, và {PB5, PB6} cho CAN. Thạch anh sử dụng cho STM32F4 là 25MHz.

- a. Cấu hình các thanh ghi RCC để cho phép CPU chạy ở tần số 168MHz.
- b. Cấu hình các thanh ghi hệ thống để sử dụng 3 chức năng UART, PWM, CAN và UART có khả năng hoạt động DMA.

Câu 4: (L.O.6) (2,5đ)

Trong cấu hình ở Câu 1, giả sử dùng STM32F4 để giải mã địa chỉ 16-bit dữ liệu và địa chỉ bắt đầu của Module 3 CS_PWM là 0x6000_0040. Hàm nhận dữ liệu từ UART được cho trước như sau:

- Hàm nhận N byte từ bộ đệm rxbuff qua UART: void receive_data (char *rxbuff, int N). Biết dữ liệu rxbuff dạng binary với tổng số byte nhận là 35 và format chuỗi dữ liệu nhận vào:

0xAA (1byte)	0x55 (1byte)	PWM1 (byteH)	PWM1 (byteL)	---	PWM16 (byteH)	PWM16 (byteL)	CRC (1byte)
-----------------	-----------------	-----------------	-----------------	-----	------------------	------------------	----------------

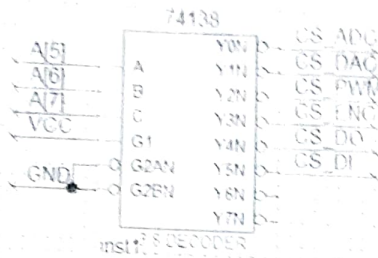
Viết chương trình đọc dữ liệu từ UART, kiểm tra chuỗi dữ liệu nếu header bằng {0xAA, 0x55} và đúng CRC thì chuyển đổi giá trị và xuất ra 16 kênh PWM, biết rằng CRC bằng XOR 34 byte dữ liệu trong chuỗi (từ byte thứ 1 đến byte thứ 34).

ĐÁP ÁN

Câu 1: (2,5đ)

- Vẽ đúng sơ đồ giải mã 74138

(0,5đ)



- Ghi đúng tầm địa chỉ cho 6 module

(1đ)

- Vẽ đúng sơ đồ kết nối 6 module

(1đ)

- Chú ý: Giải mã cho 16-bit data D[15:0].

Nếu sinh viên làm giải mã 8-bit data D[7:0] sẽ mất toàn bộ điểm câu 1

Câu 2: (2,5đ)

- Khai báo đúng module, đầu vào, đầu ra

(0,5đ)

- Xuất đúng tần số cho 2 giá trị của mode

(2đ)

Câu 3: (2,5đ)

a. (1,0đ)

- Cho phép PLL 168MHz:

$RCC_CR |= (1 << 16) | (1 << 24);$

(1đ)

$RCC_PLLCFGR |= (25 << 0) | (336 << 6) | (2 << 16)$

//cho phép HSE, PLL

//M=25, N = 336, P = 2

$$25 / M * N / P = 168$$

b. (1,5đ)

- Cấu hình thành ghi xung clock

(0,5đ)

$RCC_AHB1ENR |= (1 << 1) | (1 << 3) | (1 << 4) | (1 << 21);$ //clock cho PB, PD, PE, DMA1

$RCC_APB1ENR |= (1 << 18) | (1 << 26);$ //clock cho UART3, CAN2

$RCC_APB2ENR |= (1 << 16);$ //clock cho TIM9

- Cấu hình mode chức năng

(1đ)

$GPIOB_MODER |= (2 << 10) | (2 << 12);$

$GPIOD_MODER |= (2 << 16) | (2 << 18);$

$GPIOE_MODER |= (2 << 10) | (2 << 12);$

$GPIOB_AFRL |= (9 << 20) | (9 << 24);$

$GPIOD_AFRH |= (7 << 0) | (7 << 4);$

$GPIOE_AFRL |= (3 << 20) | (3 << 24);$

$DMA_S1CR |= (4 << 25);$

//DMA1, Stream1, Channel4, UART3_RX

$DMA_S3CR |= (4 << 25);$

//DMA1, Stream3, Channel4, UART3_TX

Câu 4: (2,5đ)


```
char rxbuff[35];
uint16_t *p;
uint8_t crc, i;

main(){
    while(1){
        receive_data(rxbuff, 35);
        crc = rxbuff[0];
        for(i=1; i<34; i++)    crc ^= rxbuff[i];
        if ((rxbuff[0]==0xAA) && (rxbuff[1]==0x55) && (rxbuff[34]==crc))
        {
            p = 0x60000040;
            for(i=1; i<17; i++)
                *p++ = (uint16_t)rxbuff[2*i]*256 + rxbuff[2*i+1];
        }
    }
}
```

Giảng viên ra đề:
(Họ và tên)

Người phê duyệt:
(Họ và tên, Chức vụ/Họ và tên)

(Ngày duyệt đề)



TRƯỜNG ĐH BÁCH KHOA – ĐHQG-HCM
KHOA ĐIỆN ĐIỆN TỬ

THI CUỐI KỲ

Học kỳ/năm học 2 2022-2023
Ngày thi 15/06/2023

Môn học Vi điều khiển và Hệ thống nhúng

Mã môn học 45173

Thời lượng 65 phút

Mã đề

Ghi chú: - Được sử dụng tài liệu

Câu 1: (3,5đ)

Thiết kế mạch giải mã địa chỉ và kết nối dữ liệu cho hệ thống (8-bit địa chỉ A7-A0; 16-bit dữ liệu D15-D0; RD, WR trên 2 chân riêng biệt) kết nối với các module bên dưới. Ghi rõ kết nối chân CS, WR, RD tới các module và địa chỉ bắt đầu, địa chỉ kết thúc của các module (không giải mã địa chỉ các kênh):

- | | |
|--|--|
| 1/ Module 1: 16 kênh ADC 16-bit | 5/ Module 5: 8 kênh I2C 16-bit |
| 2/ Module 2: 16 kênh Encoder 32-bit | 6/ Module 6: 8 kênh SPI 16-bit |
| 3/ Module 3: 8 kênh PWM 10-bit | 7/ Module 7: 8 kênh Digital Input 8-bit |
| 4/ Module 4: 4 kênh Input Capture 16-bit | 8/ Module 8: 4 kênh Digital Output 8-bit |

Câu 2: (3đ)

Trong yêu cầu thiết kế hệ thống dùng STM32F4 ta cần sử dụng Port {PD0, PD1} cho CAN, {PF0, PF1} cho I2C, {PC10, PC11} cho UART và 2 kênh DAC. Hãy cấu hình các thanh ghi hệ thống để sử dụng 3 chức năng này và I2C, DAC có khả năng hoạt động DMA với yêu cầu sau:

- Chỉ rõ chân nào sử dụng 2 kênh DAC
- Cấu hình thanh ghi cho phép xung clock để hoạt động các chức năng trên
- Cấu hình thanh ghi chân IO để lựa chọn chức năng phù hợp

Câu 3: (3,5đ)

Giả sử dùng STM32F4 giải mã địa chỉ 16-bit dữ liệu để đọc 2 kênh encoder 32-bit ở Câu 1, biết địa chỉ bắt đầu của CS_ENC là 0x6000_0020. Hàm xuất dữ liệu ra UART được cho trước như sau:

- Hàm gửi N byte từ bộ đệm *txbuff* ra UART: void *send_data* (char *txbuff, int N). Trong đó, dữ liệu *txbuff* dạng binary với tổng số byte truyền là 11 và format chuỗi dữ liệu truyền:

0x55	0xAA	ENC1	ENC2	CRC
(1byte)	(1byte)	(4bytes)	(4bytes)	(1byte)

Viết chương trình đọc 2 kênh encoder 32-bit, tính CRC và truyền dữ liệu ra UART theo frame ở trên, biết rằng CRC bằng xor 10 byte dữ liệu trong frame từ byte thứ 1 (0x55) đến byte thứ 10 (ENC2).

(Hết)

ĐÁP ÁN

Câu 1: (3,5đ)

- Vẽ đúng sơ đồ giải mã 74138, chú ý 16-bit data D[15:0] (1đ)
- Ghi đúng tầm địa chỉ cho 6 module (1đ)
- Vẽ đúng sơ đồ kết nối 6 module (1,5đ)

Câu 2: (3đ)

- Chỉ rõ chân kết nối: (0,5đ)

PD0 – PD1: CAN1 – AF9
PF0 – PF1: I2C2 – AF4, DMA1
PC10, PC11: UART3 – AF7
PA4 – PA5: DAC, DMA1

- Cấu hình thanh ghi xung clock (1đ)

RCC_AHB1ENR |= 1 | (1<<2) | (1<<3) | (1<<5) | (1<<21); //clock cho PA,PC,PD,PF,DMA1

RCC_APB1ENR |= (1<<18) | (1<<22) | (1<<25) | (1<<29); //clock cho UART3, I2C2, CAN1,
//DAC

- Cấu hình mode chức năng (1,5đ)

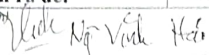
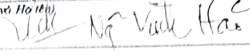
GPIOA_MODER |= (3<<8) | (3<<10);
GPIOC_MODER |= (2<<20) | (2<<22);
GPIOD_MODER |= (2<<0) | (2<<2);
GPIOF_MODER |= (2<<0) | (2<<2);

GPIOC_AFRH |= (7<<8) | (7<<12);
GPIOD_AFRH |= (9<<0) | (9<<4);
GPIOF_AFRH |= (4<<0) | (4<<4);

DMA_S2CR |= (7<<25); //DMA1, Stream2, Channel7, I2C2_RX
DMA_S7CR |= (7<<25); //DMA1, Stream7, Channel7, I2C2_TX
DMA_S5CR |= (7<<25); //DMA1, Stream5, Channel7, DAC1
DMA_S6CR |= (7<<25); //DMA1, Stream6, Channel7, DAC2

Câu 3: (3,5đ)

```
char txbuff[11];
uint16_t *p, temp, i, crc;
main(){
    while(1){
        p = 0x60000020;
        txbuff[0] = 0x55;        txbuff[1] = 0xAA;
        for(i=0; i<4; i++){
            temp = *p++;
            txbuff[2*i+2] = temp>>8;
            txbuff[2*i+3] = temp<<8;
        }
        crc = 0;
        for(i=0; i<10; i++) crc ^= txbuff[i];
        txbuff[10] = crc;
        send_data(txbuff, 11);
    }
}
```

Giảng viên ra đề: (Chữ ký và Họ tên)	(Ngày ra đề)	Người phê duyệt: (Chữ ký, Chức vụ và Họ tên)	(Ngày duyệt đề)
			

 TRƯỜNG ĐH BÁCH KHOA – ĐHQG-HCM KHOA ĐIỆN ĐIỆN TỬ	THI CUỐI KỲ		Học kỳ/năm học	1	2023-2024	
			Ngày thi	06/01/2024		
	Môn học	Vi điều khiển và Hệ thống nhúng				
	Mã môn học	45173				
Thời lượng		65 phút	Mã đề			
Ghi chú: - Được sử dụng tài liệu						

Câu 1: (3,5đ)

Thiết kế mạch giải mã địa chỉ và kết nối dữ liệu cho hệ thống (8-bit địa chỉ A7-A0; 16-bit dữ liệu D15-D0; RD, WR trên 2 chân riêng biệt) kết nối với các module bên dưới. Ghi rõ kết nối chân CS, WR, RD tới các module và địa chỉ bắt đầu, địa chỉ kết thúc của các module (không giải mã địa chỉ các kênh):

- | | |
|-------------------------------------|--|
| 1/ Module 1: 16 kênh ADC 16-bit | 5/ Module 5: 8 kênh I2C 16-bit |
| 2/ Module 2: 16 kênh Encoder 32-bit | 6/ Module 6: 8 kênh SPI 16-bit |
| 3/ Module 3: 8 kênh PWM 12-bit | 7/ Module 7: 8 kênh Digital Input 8-bit |
| 4/ Module 4: 4 kênh ICAP 16-bit | 8/ Module 8: 4 kênh Digital Output 8-bit |

Câu 2: (3đ)

Trong yêu cầu thiết kế hệ thống dùng STM32F4 ta cần sử dụng Port {PD12, PD13} cho PWM, {PF0, PF1} cho I2C, {PC10, PC11} cho UART, và 2 kênh DAC. Hãy cấu hình các thanh ghi hệ thống để sử dụng 3 chức năng này và UART, DAC có khả năng hoạt động DMA với yêu cầu sau:

- Chỉ rõ chân nào sử dụng 2 kênh DAC
- Cấu hình thanh ghi cho phép xung clock để hoạt động các chức năng trên
- Cấu hình thanh ghi chân IO để lựa chọn chức năng phù hợp

Câu 3: (3,5đ)

Dùng STM32F4 giải mã địa chỉ 16-bit dữ liệu để đọc dữ liệu các kênh encoder 32-bit như ở cấu hình Câu 1, giả sử địa chỉ bắt đầu của CS_ENC là 0x6000_0010.

Hàm xuất dữ liệu ra UART được cho trước như sau: void *send_data* (char *txbuff, int N). Trong đó N là số byte cần gửi từ bộ đệm *txbuff* ra UART

Viết chương trình đọc dữ liệu từ module Encoder, kênh 1 và 2, sau đó chuyển đổi giá trị int sang ASCII và gửi ra UART theo format chuỗi dữ liệu như sau biết rằng giá trị đọc encoder có tầm từ 0 - 200.000:

Encoder 1 (6byte)	0x20 (1byte)	Encoder 2 (6byte)	0x0D (1byte)	0x0A (1byte)
----------------------	-----------------	----------------------	-----------------	-----------------

(Hết)

Giảng viên ra đề: (Chữ ký và Họ tên)	(Ngày ra đề)	Người phê duyệt: (Chữ ký, Chức vụ và Họ tên)	(Ngày duyệt đề)
--	--------------	--	-----------------

 TRƯỜNG ĐH BÁCH KHOA – ĐHQG-HCM KHOA ĐIỆN ĐIỆN TỬ	THI CUỐI KỲ		Học kỳ/năm học	1	2023-2024
			Ngày thi	23/12/2023	
	Môn học	Hệ thống điều khiển nhúng			
	Mã môn học	EE3067			
	Thời lượng	90 phút	Mã đề		

Ghi chú: - Được sử dụng tài liệu

Câu 1: (L.O.1) (2,5đ)

Thiết kế mạch giải mã địa chỉ và kết nối dữ liệu cho hệ thống (16-bit địa chỉ A15-A0; 16-bit dữ liệu D15-D0; RD, WR trên 2 chân riêng biệt) kết nối với các module bên dưới. Ghi rõ kết nối chân CS, WR, RD tới các module và địa chỉ bắt đầu, địa chỉ kết thúc của các module (không giải mã địa chỉ các kênh):

- | | |
|------------------------------------|--|
| 1/ Module 1: 16 kênh ADC 16-bit | 4/ Module 4: 2 kênh SPI 16-bit |
| 2/ Module 2: 8 kênh Encoder 32-bit | 5/ Module 5: 4 kênh Digital Input 8-bit |
| 3/ Module 3: 16 kênh PWM 10-bit | 6/ Module 6: 8 kênh Digital Output 8-bit |

Câu 2: (L.O.2) (2,5đ)

Viết chương trình Verilog thực hiện đo tốc độ động cơ bằng cách đo thời gian 4 chu kỳ liên tiếp của tín hiệu xung encoder:

- 3 đầu vào: 2 tín hiệu xung encoder: ENCA, ENCB; 1 tín hiệu xung clock 1us: clk.
- Đầu ra dữ liệu 16 bit: D[15:0], trong đó D[15] qui định chiều quay của động cơ, D[14:0] chứa thời gian (đơn vị us) của 4 chu kỳ xung ENCA liên tiếp.
- Dữ liệu ngõ ra D[15:0] được cập nhật sau mỗi 4 chu kỳ xung encoder ENCA.

Câu 3: (L.O.5) (2,5đ)

Trong yêu cầu thiết kế hệ thống dùng STM32F4 ta cần sử dụng Port {PD8, PD9} cho UART, {PE9, PE11} cho PWM, và 2 kênh DAC. Hãy cấu hình các thanh ghi hệ thống để sử dụng 3 chức năng này và UART có khả năng hoạt động DMA với các yêu cầu sau:

- Chỉ rõ chân nào sử dụng 2 kênh DAC
- Cấu hình thanh ghi cho phép xung clock để hoạt động các chức năng trên
- Cấu hình thanh ghi chân IO để lựa chọn chức năng phù hợp

Câu 4: (L.O.6) (2,5đ)

Giả sử dùng STM32F4 giải mã địa chỉ 16-bit dữ liệu để đọc dữ liệu 2 kênh encoder 32-bit như ở cấu hình Câu 1, biết địa chỉ bắt đầu của CS_ENC là 0x6000_0020.

Hàm xuất dữ liệu ra UART được cho trước như sau: void **send_data** (char *txbuff, int N). Trong đó N là số byte cần gửi từ bộ đệm **txbuff** ra UART

Viết chương trình đọc dữ liệu từ module Encoder, kênh 1 và 2, sau đó chuyển đổi giá trị int sang ASCII và gửi ra UART theo format chuỗi dữ liệu như sau biết rằng giá trị đọc encoder có tầm từ 0 - 200.000:

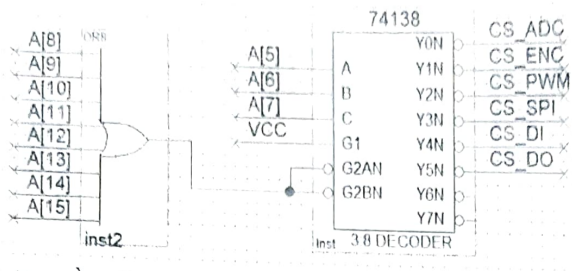
Encoder 1 (6byte)	0x20 (1byte)	Encoder 2 (6byte)	0x0D (1byte)	0x0A (1byte)
----------------------	-----------------	----------------------	-----------------	-----------------

ĐÁP ÁN

Câu 1: (2,5đ)

- Vẽ đúng sơ đồ giải mã 74138

(0.5đ)



- Ghi đúng tầm địa chỉ cho 6 module
- Vẽ đúng sơ đồ kết nối 6 module

(1đ)

(1đ)

- Chú ý: Giải mã cho 16-bit data D[15:0], 16-bit địa chỉ A[15:0].
Nếu sinh viên làm giải mã 8-bit data D[7:0] = -1đ, 8-bit địa chỉ A[7:0] = -1đ

Câu 2: (2,5đ)

- Khai báo đúng module, đầu vào, đầu ra
- Đọc đúng thời gian (đơn vị us) cho 4 chu kỳ liên tiếp ENCA
- Đọc đúng chiều quay và cập nhật ngõ ra sau 4 chu kỳ ENCA

(0.5đ)

(1đ)

(1đ)

Câu 3: (2,5đ)

- Chỉ rõ chân kết nối:

PD8 – PD9: UART3 – AF7
PE9, PE11: PWM, TIM1 – AF1
PA4 – PA5: DAC

(0.25đ)

- Cấu hình thành ghi xung clock

RCC_AHB1ENR |= (1<<0) | (1<<3) | (1<<4);
RCC_AHB1ENR |= (1<<21);

(0.75đ)
//clock cho PA, PD, PE
//clock cho DMA1

RCC_APB1ENR |= (1<<18) | (1<<29);
RCC_APB2ENR |= (1<<0);

//clock cho UART3, DAC
//clock cho PWM TIM1

- Cấu hình mode chức năng

GPIOA_MODER |= (2<<16) | (2<<18);
GPIOE_MODER |= (2<<18) | (2<<22);
GPIOA_MODER |= (3<<8) | (3<<10);

(1.5đ)

GPIOA_AFRH |= (7<<0) | (7<<4);
GPIOE_AFRH |= (1<<4) | (1<<12);

DMA_S1CR |= (4<<25);
DMA_S3CR |= (4<<25);

//DMA1, Stream1, Channel4, UART3_RX
//DMA1, Stream3, Channel4, UART3_TX

Câu 4: (2,5đ)

```
void IntToStr6(int32_t u, uint8_t *y){
    int32_t a, i;

    a = u;
    for (i=5; i>=0; i--){
        y[i] = a % 10 + 0x30;
        a = a / 10;
    }
}

main(){
    int32_t enc_value;
    uint16_t *p;
    uint8_t txbuff[15];

    p = 0x60000021;    //dia chi kenh 1 (MSB)
    enc_value = *p--;
    enc_value = enc_value<<16 + *p;
    IntToStr6(enc_value, txbuff);
    txbuff[6] = 0x20;

    p = 0x60000023;    //dia chi kenh 2 (MSB)
    enc_value = *p--;
    enc_value = enc_value<<16 + *p;
    IntToStr6(enc_value, txbuff+7);

    txbuff[13] = 0x0D;
    txbuff[14] = 0x0A;
    send_data(txbuff, 15);
}
```